VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT
on

Machine Learning (23CS6PCMAL)

Submitted by

Varun R(1BM24CS430)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
Jan-2026 to Jul-2026

B.M.S. College of Engineering,

Bull Temple Road, Bangalore 560019

(Affiliated To Visvesvaraya Technological University, Belgaum)

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "Machine Learning (23CS6PCMAL)" carried out by **Varun R(1BM24CS430),** who is bonafide student of **B.M.S. College of Engineering.** It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements in respect of an Machine Learning (23CS6PCMAL) work prescribed for the said degree.

Amrutha B Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE

Index

Sl. No.	Date	Experiment Title	Page No.
1	4-2-2026	Write a python program to import and export data using Pandas library functions	1-3
2	4-2-2026	Demonstrate various data pre-processing techniques for a given dataset	4-9
3	4-2-2026	Implement Linear and Multi-Linear Regression algorithm using appropriate dataset	10-14
4	4-2-2026	Build Logistic Regression Model for a given dataset	15-20
5	25-2-2026	Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample.	21-25
6	25-2-2026	Build KNN Classification model for a given dataset.	26-30
7	4-3-2026	Build Support vector machine model for a given dataset	31-34
8	25-3-2026	Implement Random forest ensemble method on a given dataset.	35-36
9	25-3-2026	Implement Boosting ensemble method on a given dataset.	37-39
10	1-4-2026	Build k-Means algorithm to cluster a set of data stored in a .CSV file.	40-41
11	15-4-2026	Implement Dimensionality reduction using Principal Component Analysis (PCA) method.	42-45

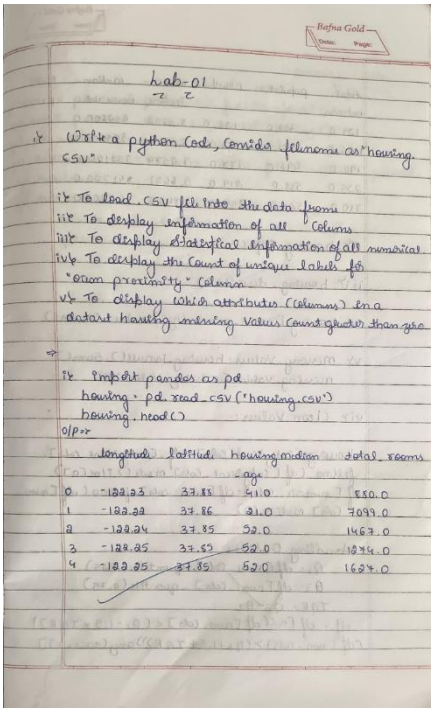
Github Link:

https://github.com/varun-bmsce/ML

Program 1

Write a python program to import and export data using Pandas library functions

Screenshot



Lab-01

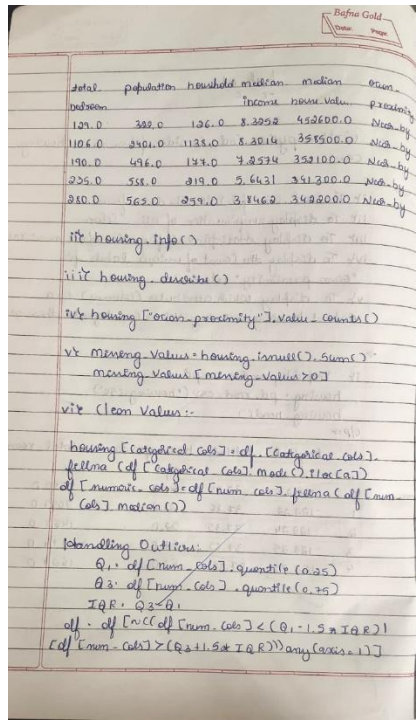
i) Write a python code, consider filename as "housing.csv"

i) To load .csv file into the data frame
ii) To display information of all columns
iii) To display statistical information of all numerical
iv) To display the count of unique labels for "ocean proximity" column
v) To display which attributes (columns) in a dataset having missing values count greater than zero

```
i) import pandas as pd
   housing = pd.read_csv("housing.csv")
   housing.head()
```

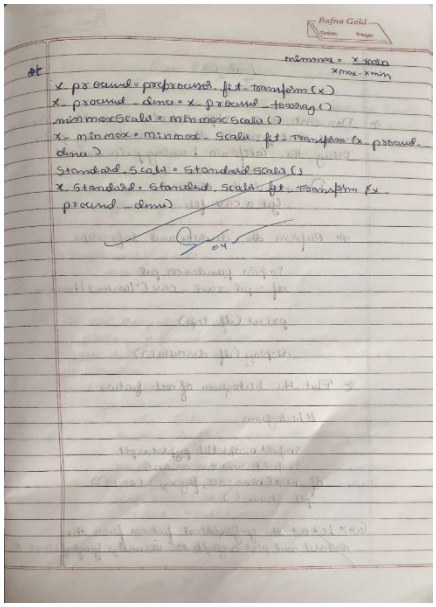
o/p:-

	longitude	latitude	housing_median_age	total_rooms
0	-122.23	37.88	41.0	880.0
1	-122.22	37.86	21.0	7099.0
2	-122.24	37.85	52.0	1467.0
3	-122.25	37.85	52.0	1274.0
4	-122.25	37.85	52.0	1627.0



total_bedroom	population	household	median_income	median_house_value	ocean_proximity
129.0	322.0	126.0	8.3252	452600.0	NEAR BAY
1106.0	2401.0	1138.0	8.3014	358500.0	NEAR BAY
190.0	496.0	177.0	7.2574	352100.0	NEAR BAY
235.0	558.0	219.0	5.6431	341300.0	NEAR BAY
280.0	565.0	259.0	3.8462	342200.0	NEAR BAY

```
iii) housing.info()

iii) housing.describe()

iv) housing["ocean_proximity"].value_counts()

v) missing_values = housing.isnull().sum()
   missing_values[missing_values>0]

vi) Clean Values:-

housing[categorical_cols] = df[categorical_cols].
fillna(df[categorical_cols].mode().iloc[0])
df[numeric_cols] = df[num_cols].fillna(df[num_cols].median())

Handling Outliers:
   Q1 = df[num_cols].quantile(0.25)
   Q3 = df[num_cols].quantile(0.75)
   IQR = Q3 - Q1
   df = df[~((df[num_cols] < (Q1 - 1.5 * IQR))
   | (df[num_cols] > (Q3 + 1.5 * IQR))).any(axis=1)]
```



```
minmax = x - xmin / xmax - xmin

x_preprocessed = preprocessed.fit_transform(x)
x_preprocessed_dense = x_preprocessed.toarray()
minmaxScaler = MinMaxScaler()
x_minmax = minmaxScaler.fit_transform(x_preprocessed_dense)
StandardScaler = StandardScaler()
x_standard = StandardScaler.fit_transform(x_preprocessed_dense)
```

Code:

diabetes

In [1]:
```
import numpy as np
import pandas as pd
```

In [2]:
```
df=pd.read_csv('/content/Dataset of Diabetes .csv')
```

In [3]:
```
df.head()
```

Out[3]:

	ID	No_Pation	Gender	AGE	Urea	Cr	HbA1c	Chol	TG	HDL	LDL	VLDL	BMI	CLASS
0	502	17975	F	50	4.7	46	4.9	4.2	0.9	2.4	1.4	0.5	24.0	N
1	735	34221	M	26	4.5	62	4.9	3.7	1.4	1.1	2.1	0.6	23.0	N
2	420	47975	F	50	4.7	46	4.9	4.2	0.9	2.4	1.4	0.5	24.0	N
3	680	87656	F	50	4.7	46	4.9	4.2	0.9	2.4	1.4	0.5	24.0	N
4	504	34223	M	33	7.1	46	4.9	4.9	1.0	0.8	2.0	0.4	21.0	N

In [7]:
```
df.describe()
```

Out[7]:

	ID	No_Pation	AGE	Urea	Cr	HbA1c	Chol	TG	HDL	LDL	VLDL	
count	1000.000000	1.000000e+03	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	10
mean	340.500000	2.705514e+05	53.528000	5.124743	68.943000	8.281160	4.862820	2.349610	1.204750	2.609790	1.854700	
std	240.397673	3.380758e+06	8.799241	2.935165	59.984747	2.534003	1.301738	1.401176	0.660414	1.115102	3.663599	
min	1.000000	1.230000e+02	20.000000	0.500000	6.000000	0.900000	0.000000	0.300000	0.200000	0.300000	0.100000	
25%	125.750000	2.406375e+04	51.000000	3.700000	48.000000	6.500000	4.000000	1.500000	0.900000	1.800000	0.700000	
50%	300.500000	3.439550e+04	55.000000	4.600000	60.000000	8.000000	4.800000	2.000000	1.100000	2.500000	0.900000	
75%	550.250000	4.538425e+04	59.000000	5.700000	73.000000	10.200000	5.600000	2.900000	1.300000	3.300000	1.500000	

```
memory usage: 109.5+ KB
```

In [4]:
```
df.isnull().sum()
```

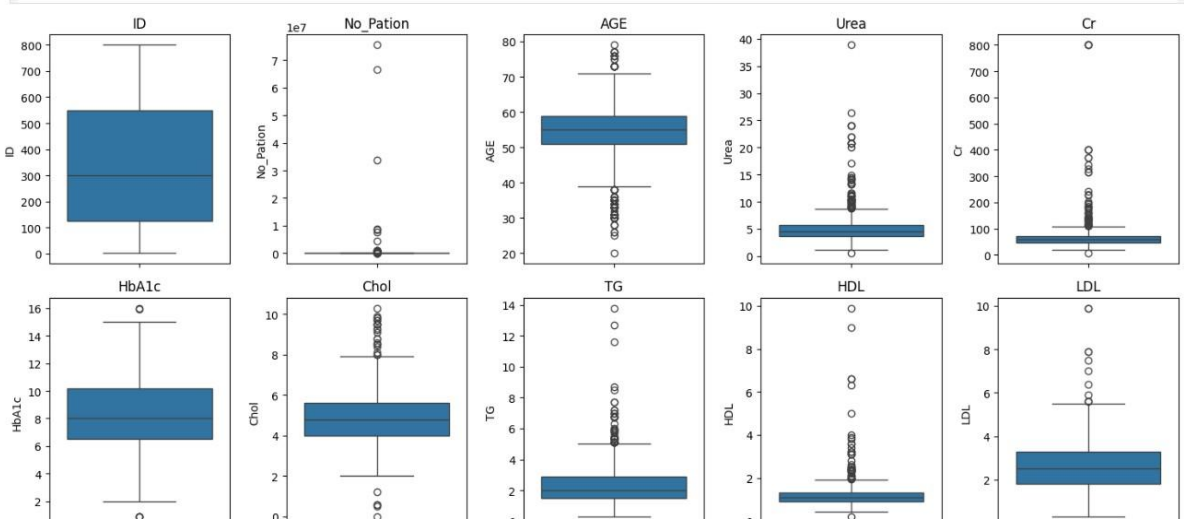
Out[4]:

	0
ID	0
No_Pation	0
Gender	0
AGE	0
Urea	0
Cr	0
HbA1c	0
Chol	0
TG	0
HDL	0
LDL	0
VLDL	0
BMI	0
CLASS	0

dtype: int64

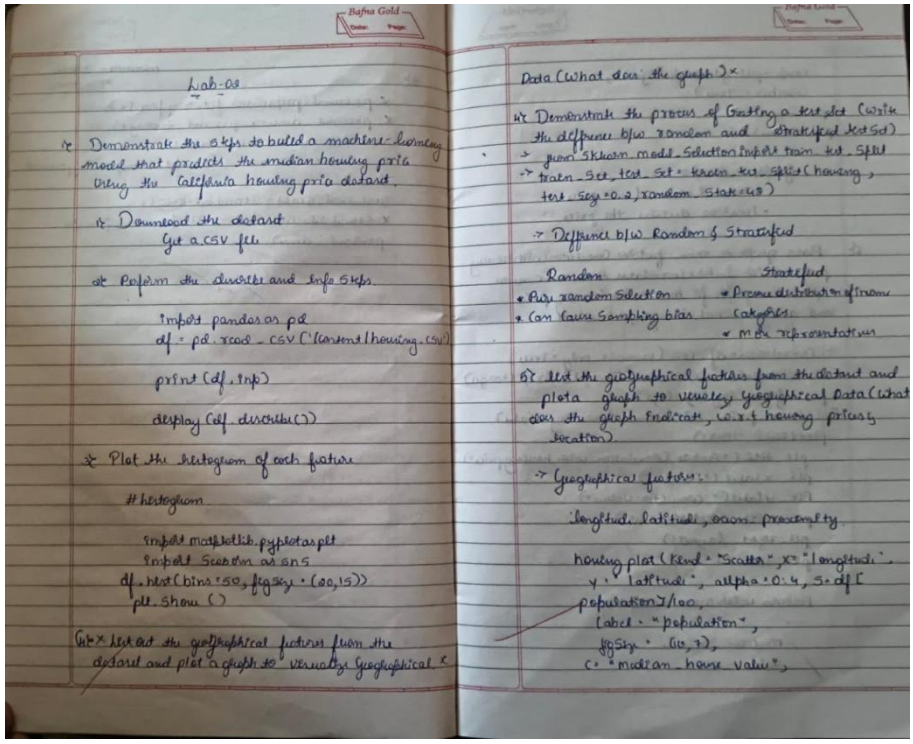
dtype: int64

In [5]:
```
import matplotlib.pyplot as plt
import seaborn as sns

# Select only numerical columns for box plots
numerical_cols = df.select_dtypes(include=np.number).columns

# Generate box plots for each numerical column
plt.figure(figsize=(15, 10))
for i, col in enumerate(numerical_cols):
    plt.subplot(3, 5, i + 1) # Adjust subplot grid as needed
    sns.boxplot(y=df[col])
    plt.title(col)
plt.tight_layout()
plt.show()
```



Program 2

Demonstrate various data pre-processing techniques for a given dataset



Lab-08

1. Demonstrate the steps to build a machine learning model that predicts the median housing price using the California housing price dataset.

1. Download the dataset
   Get a CSV file

2. Perform the describe and info steps.

```
import pandas as pd
df = pd.read_csv('/content/housing.csv')

print(df.info)

display(df.describe())
```

3. Plot the histogram of each feature

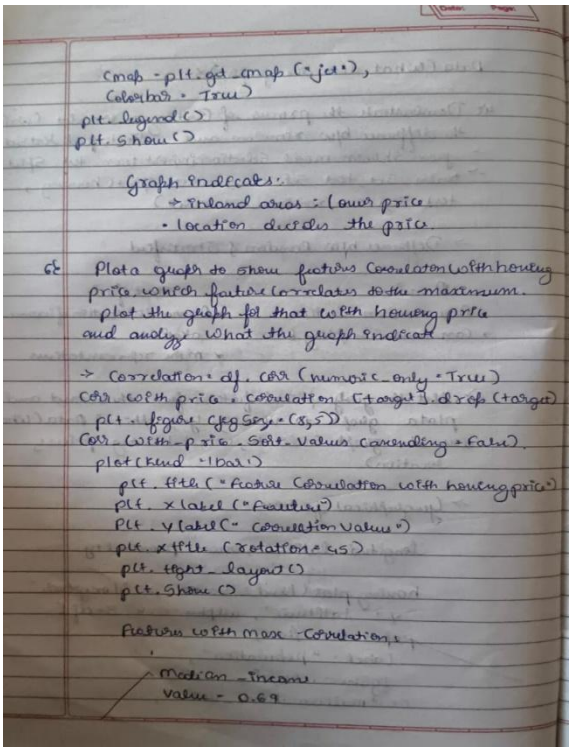
```
# histogram

import matplotlib.pyplot as plt
import seaborn as sns
df.hist(bins=50, figsize=(20,15))
plt.show()
```

Sub: Select the geographical features from the dataset and plot a graph to visualize geographical data (what does the graph indicate, w.r.t housing prices, location)

-> Geographical features:

longitude, latitude, ocean_proximity

```
housing.plot(kind="scatter", x="longitude",
y="latitude", alpha=0.4, s=df[
population]/100,
label="population",
figsize=(10,7),
c="median_house_value",
```



```
cmap=plt.get_cmap("jet"),
colorbar=True)
plt.legend()
plt.show()
```

Graph indicates:

-> Inland areas: lower price
- location decides the price.

6. Plot a graph to show features correlation with housing price. Which feature correlates to the maximum. Plot the graph for that with housing price and analyze what the graph indicate.

```
-> correlation = df.corr(numeric_only=True)
corr_with_price = correlation["target"].drop(target)
plt.figure(figsize=(8,5))
corr_with_price.sort_values(ascending=False).
plot(kind="bar")
plt.title("Feature Correlation with housing price")
plt.xlabel("Features")
plt.ylabel("Correlation Value")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

Features with max correlation:

Median_Income
value - 0.69

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, StratifiedShuffleSplit
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.base import BaseEstimator, TransformerMixin


# ========================
# 1. Load Dataset
# ========================
housing = pd.read_csv("housing.csv")
housing.head()


# ========================
# 2. Info & Describe
# ========================
housing.info()
housing.describe()


# ========================
# 3. Histograms
# ========================
housing.hist(bins=50, figsize=(20, 15))
plt.show()


# ========================
# 4. Train-Test Split
# ========================

# a) Random Split
train_set, test_set = train_test_split(
    housing,
    test_size=0.2,
```

```
    random_state=42
)

print("Training set size:", len(train_set))
print("Test set size:", len(test_set))


# b) Stratified Sampling (based on income)
housing["income_cat"] = pd.cut(
    housing["median_income"],
    bins=[0, 1.5, 3.0, 4.5, 6, np.inf],
    labels=[1, 2, 3, 4, 5]
)

print(pd.DataFrame(housing["income_cat"]))

split = StratifiedShuffleSplit(
    n_splits=1,
    test_size=0.2,
    random_state=42
)

for train_index, test_index in split.split(housing, housing["income_cat"]):
    strat_train = housing.loc[train_index]
    strat_test = housing.loc[test_index]

print("Stratified training set size:", len(strat_train))
print("Stratified test set size:", len(strat_test))


# ========================
# 5. Geographical Visualization
# ========================
housing.plot(
    kind="scatter",
    x="longitude",
    y="latitude",
    alpha=0.4,
    s=housing["population"] / 100,
    label="population",
    figsize=(10, 7),
```

```
    c="median_house_value",
    cmap=plt.get_cmap("jet"),
    colorbar=True
)

plt.legend()
plt.show()


# ==========================
#  6. Correlation Analysis
# ==========================
corr_matrix = housing.corr(numeric_only=True)
print(
    corr_matrix["median_house_value"]
    .sort_values(ascending=False)
)

housing.plot(
    kind="scatter",
    x="median_income",
    y="median_house_value",
    alpha=0.1
)
plt.show()


# ==========================
# 7. Feature Engineering
# ==========================
housing["rooms_per_household"] = housing["total_rooms"] /
housing["households"]
housing["bedrooms_per_room"] = housing["total_bedrooms"] /
housing["total_rooms"]
housing["population_per_household"] = housing["population"] /
housing["households"]

print(
    housing.corr(numeric_only=True)["median_house_value"]
    .sort_values(ascending=False)
)
```

```python
# ========================
# 8. Data Cleaning
# ========================
# Missing values
print(housing["total_bedrooms"].isnull().sum())

# Fill with median
median = housing["total_bedrooms"].median()
housing["total_bedrooms"] = housing["total_bedrooms"].fillna(median)

print(housing["total_bedrooms"].isnull().sum())


# ========================
# 9. One-Hot Encoding
# ========================
cat_cols = housing.select_dtypes(include=['object']).columns
print("Categorical columns:", cat_cols)

ohe = OneHotEncoder(drop='first', sparse_output=False)

encoded_array = ohe.fit_transform(housing[cat_cols])
encoded_cols = ohe.get_feature_names_out(cat_cols)

encoded_housing = pd.DataFrame(
    encoded_array,
    columns=encoded_cols,
    index=housing.index
)

housing_ohe = pd.concat(
    [housing.drop(cat_cols, axis=1), encoded_housing],
    axis=1
)

housing_ohe


# ========================
```

```python
# 10. Custom Transformer
# ========================
class AttrAdder(BaseEstimator, TransformerMixin):

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        return np.c_[
            X,
            X["total_rooms"] / X["households"],
            X["population"] / X["households"],
            X["total_bedrooms"] / X["total_rooms"]
        ]


# ========================
# 11. Full Pipeline
# ========================
X = housing.drop("median_house_value", axis=1)

num_cols = X.drop("ocean_proximity", axis=1).columns
cat_cols = ["ocean_proximity"]

pipeline = ColumnTransformer([
    ("num", Pipeline([
        ("add", AttrAdder()),
        ("scale", StandardScaler())
    ]), num_cols),

    ("cat", OneHotEncoder(sparse_output=False), cat_cols)
])

X_prepared = pipeline.fit_transform(X)

print("Transformed shape:", X_prepared.shape)
```

Program 3

Implement Linear and Multi-Linear Regression algorithm using appropriate dataset

4/2/26

Lab-03

Linear regression problem using matrix approach

$\therefore \beta = ((X^T X)^{-1} X^T) Y$

x_i	y_i
1	2
2	4
3	5
4	9

$$\left[\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{bmatrix}\right]^{-1} = \begin{bmatrix} 4 & 10 \\ 10 & 30 \end{bmatrix}$$

$$(X^T X)^{-1} = \begin{bmatrix} 4 & 10 \\ 10 & 30 \end{bmatrix}^{-1} = \begin{bmatrix} 1 & 0.5 & 0 & -0.5 \\ -0.3 & -0.1 & 0.1 & 0.3 \end{bmatrix}$$

$$(X^T X)^{-1} X^T) y = \begin{bmatrix} -0.5 \\ 2.2 \end{bmatrix} \begin{matrix} a_0 \\ a_1 \end{matrix}$$

$y = -0.5 + 2.2x$

Linear - regression without matrix

x	y	x^2	xy
8	10	64	80
10	13	100	130
12	16	144	192

$\bar{x} = \frac{8+10+12}{3} = 10$

$\bar{y} = \frac{10+13+16}{3} = 39/3 = 13$

$\bar{x^2} = 308/3 = 102.64$

$a_0 = \bar{y} - a_1(\bar{x})$

$= 13 - (1.4981)(10)$

$= -1.481$

$a_1 = \frac{\bar{xy} - \bar{x}\bar{y}}{\bar{x^2} - (\bar{x})^2}$

$= \frac{134 - (13)(10)}{102.64 - (10)^2}$

$= 6/2.67 = 1.4981$

$y = a_0 + a_1(x)$

$= -1.981 + 1.4981(20)$

$= 27.991$

Code:

```
import pandas as pd
import math
# Load dataset
data = pd.read_csv("canada_per_capita_income.csv")
# Display first few rows
print(data.head())
# Extract x (year) and y (income)
x = data['year'].values
y = data['per capita income (US$)'].values
n = len(x)
```

```
14
```

```
# Calculate required sums
sum_x = sum(x)
sum_y = sum(y)
sum_xy = sum(x * y)
sum_x2 = sum(x * x)
# Calculate slope (m) and intercept (c)
m = (n * sum_xy - sum_x * sum_y) / (n * sum_x2 - sum_x**2)
c = (sum_y - m * sum_x) / n
print("Slope (m):", m)
print("Intercept (c):", c)
# Regression equation
print("Regression Model: y = mx + c")
print(f"y = {m}x + {c}")
# Predict for year 2020
year = 2020
predicted_income = m * year + c
print("Predicted per capita income for 2020:", predicted_income)

''' Predict Salary of the employee. Use the data file salary.csv file. If
required, apply the necessary data processing steps. Using this build a
regression model and predict the salary of the employee with 12 years of
experience.
'''
import pandas as pd
data = pd.read_csv("salary.csv")
print(data.head())
print(data.columns)
```

```
print(data.isnull().sum())
# Remove missing values
data = data.dropna()
x = data['YearsExperience'].astype(float).values
y = data['Salary'].astype(float).values
n = len(x)
sum_x = sum(x)

15

sum_y = sum(y)
sum_xy = sum(x * y)
sum_x2 = sum(x * x)
m = (n*sum_xy - sum_x*sum_y) / (n*sum_x2 - sum_x**2)
c = (sum_y - m*sum_x) / n
# Predict salary for 12 years of experience
yearsOfExperience = 12
predicted_salary = m * yearsOfExperience + c
print("Predicted salary for 12 years of experience:", predicted_salary)

''' Considering the data file hiring.csv. The file contains hiring
statics for a firm such as experience of candidate, his written test
score and personal interview score. Based on these 3 factors, HR will
decide the salary. Given this data, you need to build a Multiple Linear
Regression model for HR department that can help them decide salaries for
future candidates. Using this predict salaries for following candidates,
2 yr experience, 9 test score, 6 interview score
12 yr experience, 10 test score, 10 interview score
'''
import pandas as pd
import numpy as np
pd.set_option('future.no_silent_downcasting', True)
data  = pd.read_csv("hiring.csv")
print(data.head())
# Fill missing values
data['experience']  =  data['experience'].fillna('zero')
data['test_score'] = data['test_score'].fillna(data['test_score'].mean())
# Convert words to numbers
word_to_num = {
'zero':0,'one':1,'two':2,'three':3,'four':4,'five':5,
'six':6,'seven':7,'eight':8,'nine':9,'ten':10,'eleven':11
```

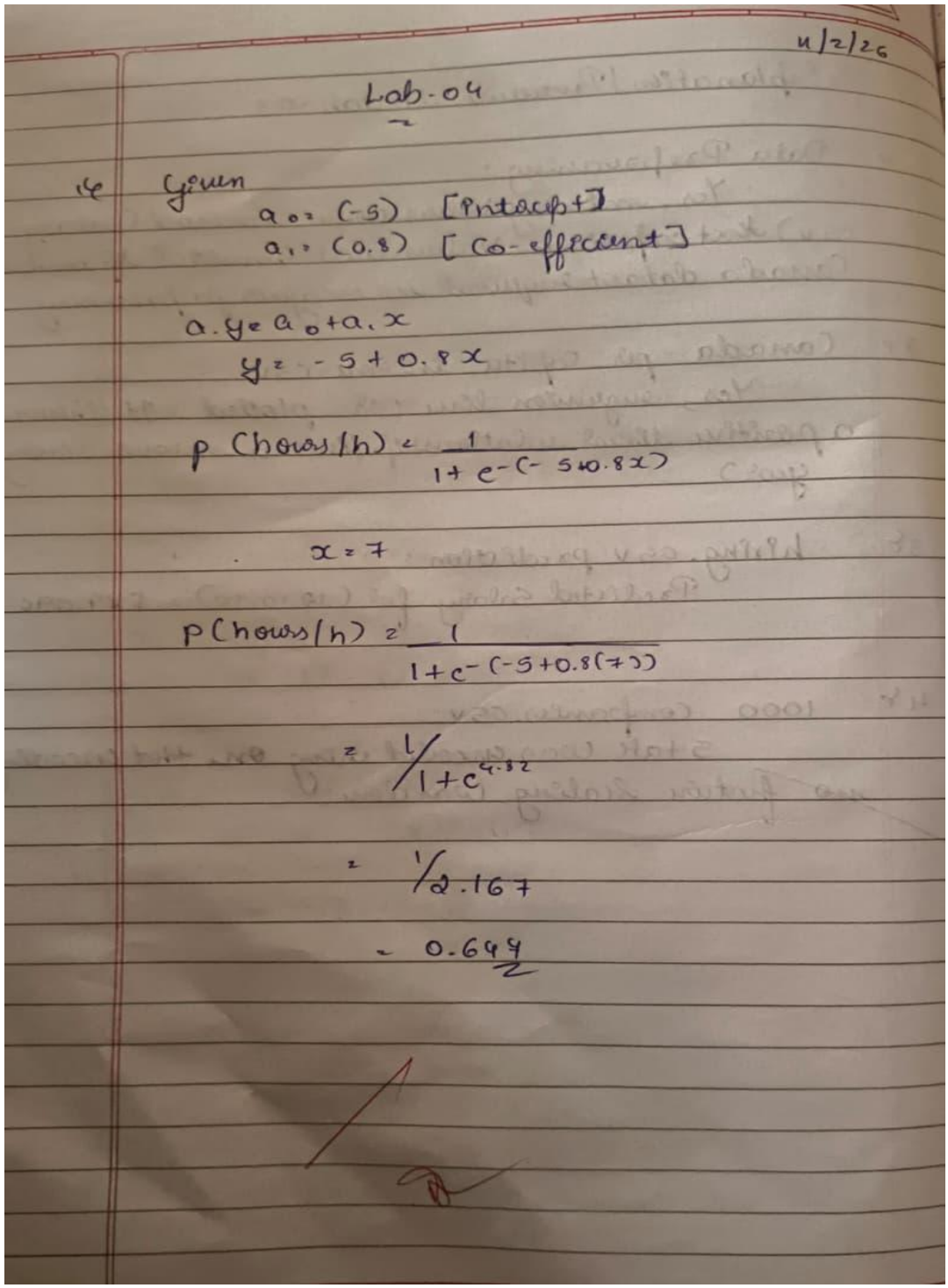
```
}
data['experience'] = data['experience'].replace(word_to_num)
# Explicit numeric conversion (prevents warning)
data['experience'] = data['experience'].astype(float)
# Convert all to numeric
data['experience'] = data['experience'].astype(float)
data['test_score'] = data['test_score'].astype(float)

16

data['interview_score'] = data['interview_score'].astype(float)
data['salary'] = data['salary'].astype(float)
# Prepare X and Y
X = data[['experience','test_score','interview_score']].values
Y = data['salary'].values
n = len(X)
# Add bias column
X_b = np.c_[np.ones((n,1)), X]
# Normal Equation
B = np.linalg.inv(X_b.T.dot(X_b)).dot(X_b.T).dot(Y)
print("Coefficients:", B)
# Predict for (12,10,10)
x_new = np.array([1,12,10,10])
predicted_salary = x_new.dot(B)
print("Predicted Salary:", predicted_salary)

''' Considering the data file 1000_companies.csv. The file contains
profit statics for a firm such as R&D Spend, Administration, Marketing
Spend and State. Based on these four factors build a Multiple Linear
Regression model to predict the profit. Using this predict profit for
following, 91694.48 R&D Spend, 515841.3 Administration, 11931.24
Marketing Spend, Florida State
'''
import pandas as pd
import numpy as np
# Load dataset
data = pd.read_csv("1000_Companies.csv")
print("Original Data:")
print(data.head())
# --- Encode categorical variable: True=1, False=0 ---
data_encoded = pd.get_dummies(data, columns=['State'], drop_first=True)
```

```
# Ensure boolean columns are converted to 0/1
for col in data_encoded.columns:
if data_encoded[col].dtype == 'bool':
data_encoded[col] = data_encoded[col].astype(int)

17

print("\nEncoded Data:")
print(data_encoded.head())
# Separate features and target
X = data_encoded.drop('Profit', axis=1)
Y = data_encoded['Profit'].values
# --- Feature scaling (manual standardization) ---
X_scaled = (X - X.mean()) / X.std()
# Convert to numpy
X_vals = X_scaled.values
n = len(X_vals)
# Add bias (intercept) column
X_b = np.c_[np.ones((n,1)), X_vals]
# --- Normal Equation for Linear Regression ---
XT = X_b.T
B = np.linalg.inv(XT.dot(X_b)).dot(XT).dot(Y)
print("\nModel coefficients:")
print(B)
```

Program 4

Build Logistic Regression Model for a given dataset

Screenshot



11/2/26

Lab-04

1) Given

$a_0 = (-5)$ [Intercept]

$a_1 = (0.8)$ [Co-efficient]

a. $y = a_0 + a_1 x$

$y = -5 + 0.8x$

$p(\text{hours}/h) = \frac{1}{1 + e^{-(-5+0.8x)}}$

$x = 7$

$p(\text{hours}/h) = \frac{1}{1 + e^{-(-5+0.8(7))}}$

$= 1 / 1 + e^{0.82}$

$= 1/2.167$

$= 0.644$

Code:

```
import pandas as pd
from matplotlib import pyplot as plt
import math
# Load dataset
df = pd.read_csv("HR_comma_sep.csv")
df.head()
'''step1: Do some exploratory data analysis to figure out which variables
have direct and clear impact on employee retention
(i.e. whether they leave the company or continue to work)
'''
# ==============================
# 1. Check Overall Leaving Rate
# ==============================
print("\nOverall Leaving Rate:")
print(df['left'].value_counts())
print("Leaving Percentage:", df['left'].mean())
# ==============================
# 2. Salary vs Retention
# ==============================
print("\nSalary vs Leaving Rate:")
salary_impact = df.groupby('salary')['left'].mean()
print(salary_impact)
salary_impact.plot(kind='bar')
plt.title("Salary vs Leaving Rate")
plt.ylabel("Leaving Rate")
plt.show()
# ==============================
# 3. Department vs Retention
# ==============================
print("\nDepartment vs Leaving Rate:")
dept_impact = df.groupby('Department')['left'].mean()

20

print(dept_impact)
dept_impact.plot(kind='bar', figsize=(10,5))
plt.title("Department vs Leaving Rate")
plt.xticks(rotation=45)
plt.ylabel("Leaving Rate")
plt.show()
```

```
# ==============================
# 4. Satisfaction Level Impact
# ==============================
print("\nAverage Satisfaction (Left vs Stayed):")
print(df.groupby('left')['satisfaction_level'].mean())
plt.scatter(df['satisfaction_level'], df['left'])
plt.title("Satisfaction Level vs Leaving")
plt.xlabel("Satisfaction Level")
plt.ylabel("Left (1=Yes, 0=No)")
plt.show()
# ==============================
# 5. Monthly Hours Impact
# ==============================
print("\nAverage Monthly Hours (Left vs Stayed):")
print(df.groupby('left')['average_montly_hours'].mean())
# ==============================
# 6. Time Spent in Company Impact
# ==============================
print("\nTime Spent in Company (Left vs Stayed):")
print(df.groupby('left')['time_spend_company'].mean())
# ==============================
# 7. Promotion Impact
# ==============================
print("\nPromotion vs Leaving Rate:")
print(df.groupby('promotion_last_5years')['left'].mean())
# step 2: Plot bar charts showing impact of employee salaries on
retention
salary_left = df.groupby('salary')['left'].mean()
print(salary_left)

21

salary_left.plot(kind='bar')
plt.title("Salary vs Leaving Rate")
plt.show()
#step 3: Plot bar charts showing corelation between department and
employee retention
dept_left = df.groupby('Department')['left'].mean()
print(dept_left)
dept_left.plot(kind='bar', figsize=(10,5))
plt.xticks(rotation=45)
```

```
plt.title("Department vs Leaving Rate")
plt.show()
# step 4: Build logistic regression model using variables that were
narrowed down in step 1
# ========================================================
# 1. Select Feature for Manual Logistic Regression
# (Using satisfaction_level)
# ========================================================
X = list(df['satisfaction_level'])
y = list(df['left'])
n = len(X)
# ========================================================
# 2. Define Sigmoid Function
# ========================================================
def sigmoid(z):
return 1 / (1 + math.exp(-z))
# ========================================================
# 3. Initialize Parameters
# ========================================================
w = -5 # weight
b = 2 # bias
learning_rate = 0.1
epochs = 200
# ========================================================
# 4. Manual Gradient Descent Training
# ========================================================

22

for epoch in range(epochs):
dw = 0
db = 0
for i in range(n):
z = w * X[i] + b
y_pred = sigmoid(z)
dw += (y_pred - y[i]) * X[i]
db += (y_pred - y[i])
dw = dw / n
db = db / n
w = w - learning_rate * dw
b = b - learning_rate * db
```

```
print("\nFinal Weight:", w)
print("Final Bias:", b)
# ========================================================
# 5. Prediction Function
# ========================================================
def predict(x):
z = w * x + b
prob = sigmoid(z)
if prob >= 0.5:
return 1
else:
return 0
# step 5: Measure the accuracy of the model
correct = 0
for i in range(n):
if predict(X[i]) == y[i]:
correct += 1
accuracy = correct / n
print("\nModel Accuracy:", accuracy)
```

Logistic Regression (Multiclass Classification)

23

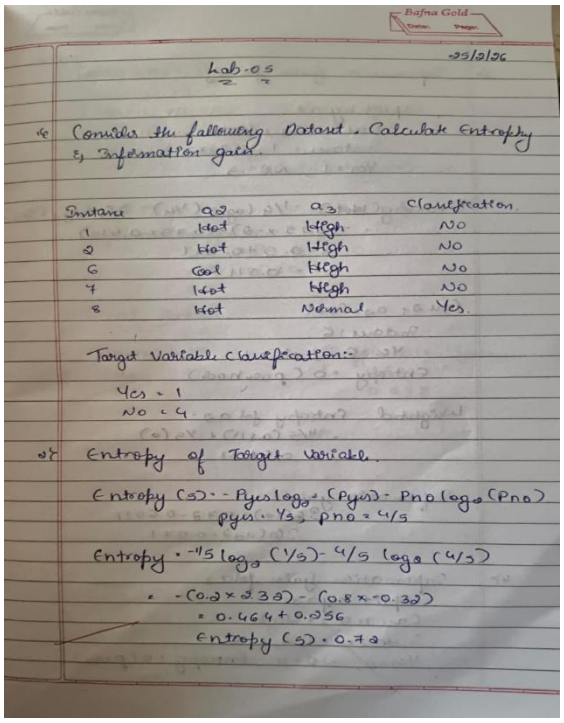
```
# ========================================================
# 1. Import Libraries
# ========================================================
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn import metrics
import matplotlib.pyplot as plt
# ========================================================
# 2. Load Dataset
# ========================================================
zoo = pd.read_csv("zoo-data.csv")
class_type = pd.read_csv("zoo-class-type.csv")
print("First 5 rows of Zoo Dataset:")
print(zoo.head())
# ========================================================
```

```
# 3. Data Preprocessing
# Remove animal name column (not useful for prediction)
zoo = zoo.drop('animal_name', axis=1)
# Features and Target
X = zoo.drop('class_type', axis=1)
y = zoo['class_type']

# 4. Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42
)
# ========================================================
# 5. Build Multiclass Logistic Regression Model
# ========================================================
model = LogisticRegression(max_iter=2000)

model.fit(X_train, y_train)
# ========================================================
# 6. Prediction
# ========================================================
y_pred = model.predict(X_test)
# ========================================================
# 7. Accuracy
# ========================================================
accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy of Multiclass Logistic Regression Model:", accuracy)
# ========================================================
# 8. Confusion Matrix
# ========================================================
# Create confusion matrix
conf_matrix = metrics.confusion_matrix(y_test, y_pred)
# Get unique labels present in test set
labels = sorted(y_test.unique())
# Display confusion matrix correctly
cm_display = metrics.ConfusionMatrixDisplay(
confusion_matrix=conf_matrix,
display_labels=labels
)
cm_display.plot()
plt.title("\nConfusion Matrix")
plt.show()
```

Program 5

Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample.

Screenshot



Lab-05

25/2/26

1) Consider the following Dataset. Calculate Entropy & Information gain.

Instance	a2	a3	Classification
1	Hot	High	No
2	Hot	High	No
6	Cool	High	No
7	Hot	High	No
8	Hot	Normal	Yes

Target Variable Classification:

Yes = 1
No = 4

2) Entropy of Target variable.

Entropy (S) = $-P_{yes}\log_2(P_{yes}) - P_{no}\log_2(P_{no})$

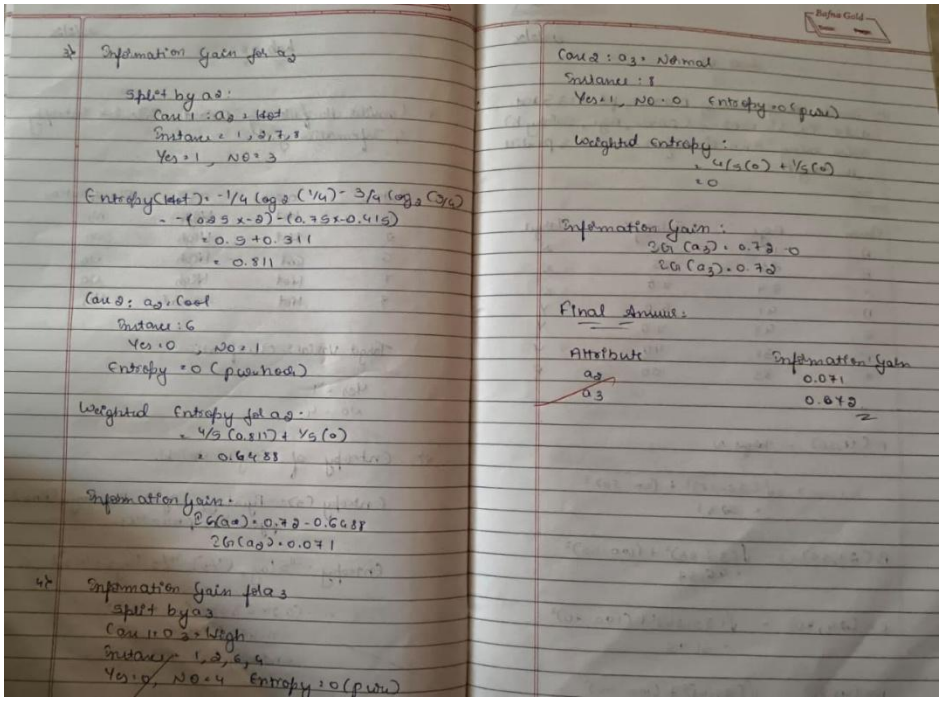
$P_{yes} = 1/5$, $P_{no} = 4/5$

Entropy = $-1/5 \log_2(1/5) - 4/5 \log_2(4/5)$

$= -(0.2 \times -2.32) - (0.8 \times -0.32)$

$= 0.464 + 0.256$

Entropy (S) = 0.72



3) Information Gain for a_2

Split by a2:

Case 1: a_2 = Hot

Instance = 1, 2, 7, 8

Yes = 1, No = 3

Entropy(Hot) = $-1/4 (\log_2(1/4)) - 3/4 \log_2(3/4)$

$= -(0.25 \times -2) - (0.75 \times -0.415)$

$= 0.5 + 0.311$

$= 0.811$

Case 2: a_2 = Cool

Instance: 6

Yes = 0, No = 1

Entropy = 0 (pure node)

Weighted Entropy for a_2:

$= 4/5 (0.811) + 1/5 (0)$

$= 0.6488$

Information gain:

$IG(a_2) = 0.72 - 0.6488$

$IG(a_2) = 0.071$

4) Information Gain for a_3

Split by a_3

Case 1: a_3 = High

Instance: 1, 2, 6, 7

Yes = 0, No = 4 Entropy = 0 (pure)

Case 2: a_3 = Normal

Instance: 8

Yes = 1, No = 0 Entropy = 0 (pure)

Weighted Entropy:

$= 4/5(0) + 1/5(0)$

$= 0$

Information Gain:

$IG(a_2) = 0.72 - 0$

$IG(a_3) = 0.72$

Final Answer:

Attribute	Information Gain
a2	0.071
a3	0.072

Code:

```
# ==========================
# Decision Tree - IRIS Dataset
# ==========================
import pandas as pd
import numpy as np
import math
import matplotlib.pyplot as plt

from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor,
plot_tree
from sklearn.model_selection import train_test_split

# Load dataset
df = pd.read_csv("iris (1).csv")

# Encode target manually
target_column = df.columns[-1]
classes = df[target_column].unique()
class_map = {c: i for i, c in enumerate(classes)}
df[target_column] = df[target_column].map(class_map)

# Split features and target
X = df.drop(target_column, axis=1)
y = df[target_column]

# Train-test split (80-20)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train model
clf = DecisionTreeClassifier(criterion='entropy')
clf.fit(X_train, y_train)

# Predict
y_pred = clf.predict(X_test)

# -------- Manual Accuracy --------
correct = 0
for i in range(len(y_test)):
```

```
    if y_test.iloc[i] == y_pred[i]:
        correct += 1

accuracy = correct / len(y_test)
print("Accuracy:", accuracy)

# -------- Manual Confusion Matrix --------
num_classes = len(classes)
conf_matrix = np.zeros((num_classes, num_classes))

for t, p in zip(y_test, y_pred):
    conf_matrix[int(t)][int(p)] += 1

print("Confusion Matrix:")
print(conf_matrix)

# ------------------- Plot Tree -------------------
plt.figure(figsize=(10, 8))
plot_tree(
    clf,
    filled=True,
    feature_names=X.columns,
    class_names=[str(c) for c in classes]
)
plt.show()


# ==========================
# Decision Tree - Drug Dataset
# ==========================
df = pd.read_csv("drug.csv")

# Encode categorical columns
for col in df.columns:
    if df[col].dtype == object:
        unique_vals = df[col].unique()
        mapping = {v: i for i, v in enumerate(unique_vals)}
        df[col] = df[col].map(mapping)

# Split features and target
target_column = df.columns[-1]
```

```python
X = df.drop(target_column, axis=1)
y = df[target_column]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train model
clf = DecisionTreeClassifier(criterion='entropy')
clf.fit(X_train, y_train)

# Predict
y_pred = clf.predict(X_test)

# -------- Manual Accuracy --------
correct = 0
for i in range(len(y_test)):
    if y_test.iloc[i] == y_pred[i]:
        correct += 1

accuracy = correct / len(y_test)
print("Accuracy:", accuracy)

# -------- Manual Confusion Matrix --------
num_classes = len(y.unique())
conf_matrix = np.zeros((num_classes, num_classes))

for t, p in zip(y_test, y_pred):
    conf_matrix[int(t)][int(p)] += 1

print("Confusion Matrix:")
print(conf_matrix)

# ------------------- Plot Tree -------------------
plt.figure(figsize=(10, 8))
plot_tree(
    clf,
    filled=True,
    feature_names=X.columns
)
```

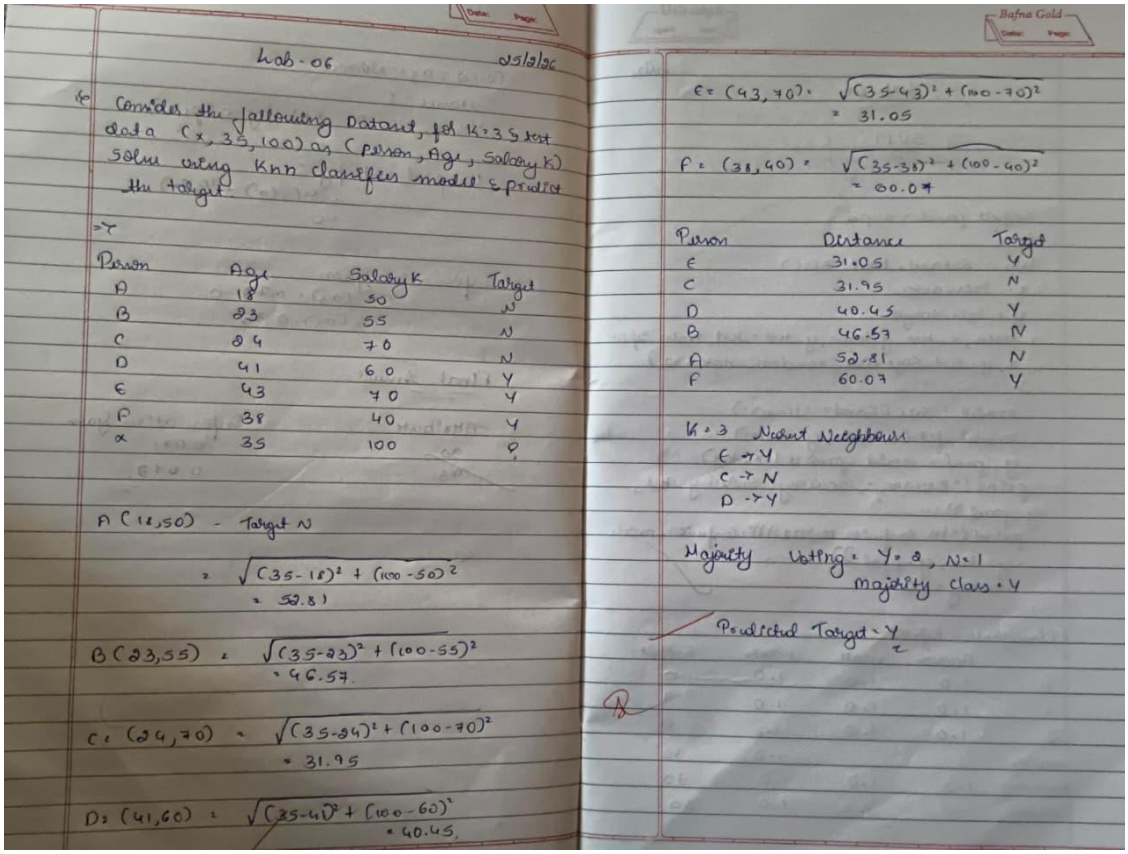
```python
plt.show()


# ==========================
# Regression Tree - Petrol Consumption
# ==========================
df = pd.read_csv("petrol_consumption.csv")

# Split features and target
target_column = df.columns[-1]
X = df.drop(target_column, axis=1)
y = df[target_column]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train model
reg = DecisionTreeRegressor()
reg.fit(X_train, y_train)

# Predict
y_pred = reg.predict(X_test)

# ------------------ Manual MAE ------------
mae = 0
for i in range(len(y_test)):
    mae += abs(y_test.iloc[i] - y_pred[i])
mae = mae / len(y_test)

# ------------------ Manual MSE ------------
mse = 0
for i in range(len(y_test)):
    mse += (y_test.iloc[i] - y_pred[i]) ** 2
mse = mse / len(y_test)

# ------------------ Manual RMSE ------------------
rmse = math.sqrt(mse)

print("Mean Absolute Error:", mae)
```

```
print("Mean Squared Error:", mse)
```

Program 6

Build KNN Classification model for a given dataset

Screenshot



Lab - 06 25/2/26

Q. Consider the following Dataset, for K=3 & test data (x, 35, 100) as (person, Age, Salary K) solve using Knn classifier model & predict the target.

=>

Person	Age	Salary K	Target
A	18	50	N
B	23	55	N
C	34	70	N
D	41	60	Y
E	43	70	Y
F	38	40	Y
x	35	100	?

A (18,50) - Target N

$= \sqrt{(35-18)^2 + (100-50)^2}$
$= 52.81$

B (23,55) $= \sqrt{(35-23)^2 + (100-55)^2}$
$= 46.57$

C = (34,70) $= \sqrt{(35-34)^2 + (100-70)^2}$
$= 31.95$

D = (41,60) $= \sqrt{(35-41)^2 + (100-60)^2}$
$= 40.45$

E = (43,70) $= \sqrt{(35-43)^2 + (100-70)^2}$
$= 31.05$

F = (38,40) $= \sqrt{(35-38)^2 + (100-40)^2}$
$= 60.07$

Person	Distance	Target
E	31.05	Y
C	31.95	N
D	40.45	Y
B	46.57	N
A	52.81	N
F	60.07	Y

K = 3 Nearest Neighbours
E → Y
C → N
D → Y

Majority Voting = Y = 2, N = 1
majority class = Y

Predicted Target = Y

```
# ========================
# KNN on IRIS Dataset
# ========================
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from collections import Counter
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    confusion_matrix,
```

```
    classification_report,
    accuracy_score
)

# Load dataset
data = pd.read_csv("iris (1).csv")

# Features and target
X = data.iloc[:, :-1].values
y = data.iloc[:, -1].values

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Euclidean Distance
def euclidean_distance(a, b):
    return np.sqrt(np.sum((a - b) ** 2))

# KNN Function
def knn_predict(X_train, y_train, test_point, k):
    distances = []

    for i in range(len(X_train)):
        dist = euclidean_distance(test_point, X_train[i])
        distances.append((dist, y_train[i]))

    distances.sort(key=lambda x: x[0])
    neighbors = [distances[i][1] for i in range(k)]

    return Counter(neighbors).most_common(1)[0][0]

# Choose k
k = 5

# Predictions
predictions = []
for test_point in X_test:
    predictions.append(knn_predict(X_train, y_train, test_point, k))
```

```python
# Accuracy
print("Accuracy:", accuracy_score(y_test, predictions))

# Confusion Matrix
cm = confusion_matrix(y_test, predictions)
plt.figure()
sns.heatmap(cm, annot=True, fmt='d')
plt.title("Confusion Matrix - IRIS")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

# Classification Report
print("\nClassification Report:\n")
print(classification_report(y_test, predictions))


# ========================
# KNN on Diabetes Dataset
# ========================
from sklearn.preprocessing import StandardScaler

# Load dataset
data = pd.read_csv("diabetes.csv")

X = data.iloc[:, :-1].values
y = data.iloc[:, -1].values

# Feature Scaling
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Reuse same functions (euclidean_distance, knn_predict)

k = 5
```

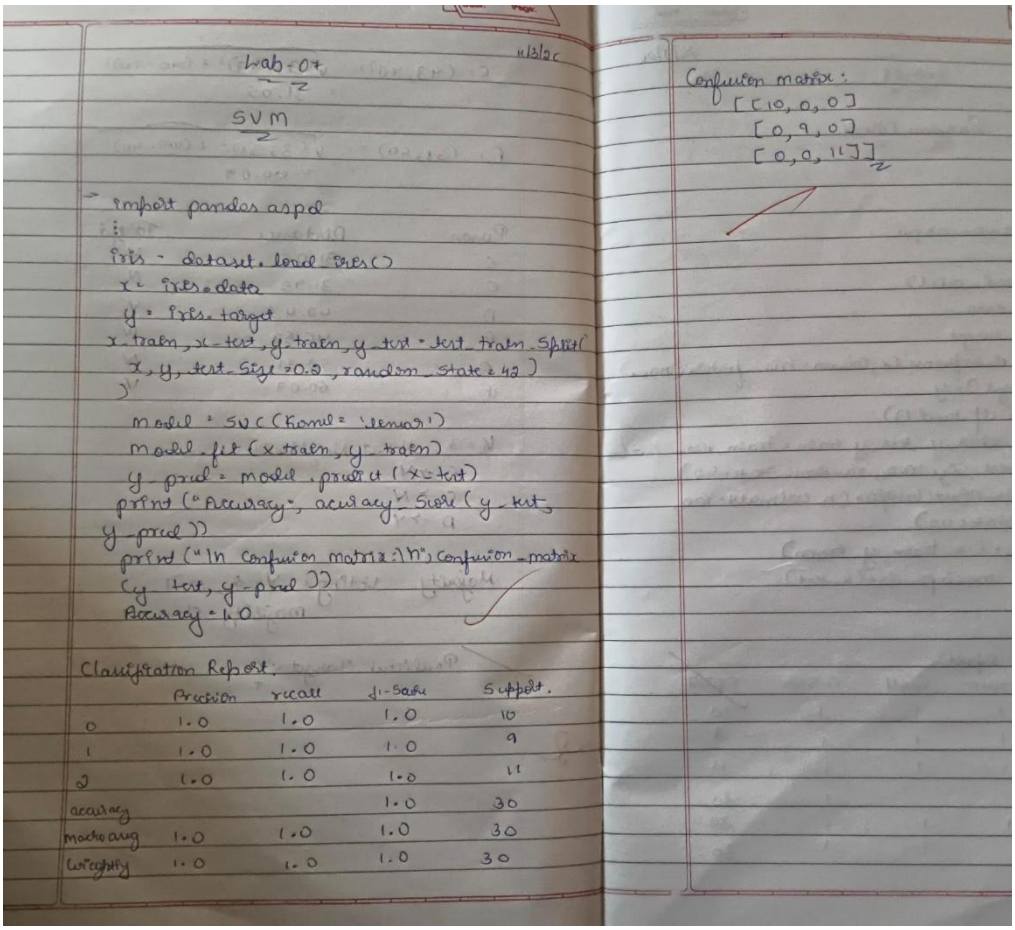
```
predictions = []
for test_point in X_test:
    predictions.append(knn_predict(X_train, y_train, test_point, k))

print("Accuracy:", accuracy_score(y_test, predictions))

# Confusion Matrix
cm = confusion_matrix(y_test, predictions)
plt.figure()
sns.heatmap(cm, annot=True, fmt='d')
plt.title("Confusion Matrix - Diabetes")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()


# ========================
# KNN on Heart Dataset (Best K)
# ========================
# Load dataset
data = pd.read_csv("heart.csv")

X = data.iloc[:, :-1].values
y = data.iloc[:, -1].values

# Feature Scaling
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Find Best K
best_k = 1
best_accuracy = 0

for k in range(1, 21):
    predictions = []
```

```
    for test_point in X_test:
        predictions.append(knn_predict(X_train, y_train, test_point, k))

    acc = accuracy_score(y_test, predictions)

    if acc > best_accuracy:
        best_accuracy = acc
        best_k = k

print("Best K:", best_k)
print("Best Accuracy:", best_accuracy)

# Final Predictions
predictions = []
for test_point in X_test:
    predictions.append(knn_predict(X_train, y_train, test_point, best_k))

# Confusion Matrix
cm = confusion_matrix(y_test, predictions)
plt.figure()
sns.heatmap(cm, annot=True, fmt='d')
plt.title("Confusion Matrix - Heart Dataset")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

# Classification Report
print("\nClassification Report:\n")
print(classification_report(y_test, predictions))
```

Program 7

Build Support vector machine model for a given dataset

Screenshot



Lab - 07

SVM

```
import pandas as pd
iris = dataset.load_iris()
x = iris.data
y = iris.target
x_train, x_test, y_train, y_test = test_train_split(
  x, y, test_size = 0.3, random_state = 42)
model = SVC(kernel = 'linear')
model.fit(x_train, y_train)
y_pred = model.predict(x_test)
print("Accuracy", accuracy_score(y_test,
y_pred))
print("\n Confusion matrix:\n", confusion_matrix
(y_test, y_pred))
Accuracy = 1.0
```

Classification Report:

	Precision	recall	f1-score	Support.
0	1.0	1.0	1.0	10
1	1.0	1.0	1.0	9
2	1.0	1.0	1.0	11
accuracy			1.0	30
macro avg	1.0	1.0	1.0	30
weighted	1.0	1.0	1.0	30

Confusion matrix:

```
[[10, 0, 0]
 [0, 9, 0]
 [0, 0, 11]]
```

Code:

```
# ============================================================
# SVM Classification for IRIS Dataset
# ============================================================
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix
```

```python
# Load dataset
data = pd.read_excel("iris.xlsx")

# Features and target
X = data.iloc[:, :-1]
y = data.iloc[:, -1]

# Train-Test split (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ----------------------------------------------------------------
# Linear Kernel SVM
# ----------------------------------------------------------------
linear_svm = SVC(kernel='linear')
linear_svm.fit(X_train, y_train)

y_pred_linear = linear_svm.predict(X_test)

print("Linear Kernel Accuracy:", accuracy_score(y_test, y_pred_linear))

cm_linear = confusion_matrix(y_test, y_pred_linear)

plt.figure()
sns.heatmap(cm_linear, annot=True, fmt='d', cmap='Blues')
plt.title("Confusion Matrix - Linear Kernel")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

# ----------------------------------------------------------------
# RBF Kernel SVM
# ----------------------------------------------------------------
rbf_svm = SVC(kernel='rbf')
rbf_svm.fit(X_train, y_train)

y_pred_rbf = rbf_svm.predict(X_test)

print("RBF Kernel Accuracy:", accuracy_score(y_test, y_pred_rbf))
```

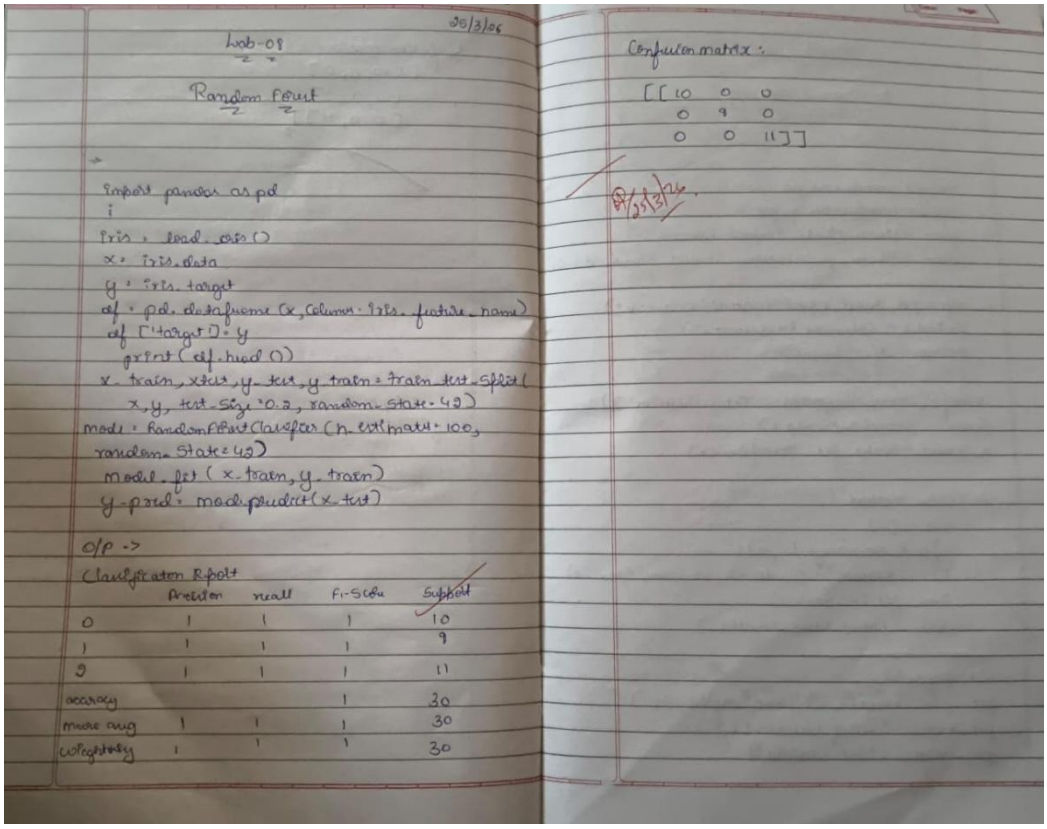
```
cm_rbf = confusion_matrix(y_test, y_pred_rbf)

plt.figure()
sns.heatmap(cm_rbf, annot=True, fmt='d', cmap='Greens')
plt.title("Confusion Matrix - RBF Kernel")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()


# ============================================================
# SVM Classification for Letter Recognition Dataset
# ============================================================
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.preprocessing import label_binarize
from sklearn.metrics import accuracy_score, confusion_matrix, roc_curve, auc
from sklearn.multiclass import OneVsRestClassifier

# Load dataset
data = pd.read_excel("letter-recognition.xlsx")

# Features and target
X = data.iloc[:, 1:]
y = data.iloc[:, 0]

# Binarize labels
classes = sorted(y.unique())
y_bin = label_binarize(y, classes=classes)

# Train-Test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y_bin, test_size=0.2, random_state=42
)

# Train SVM (One-vs-Rest)
model = OneVsRestClassifier(SVC(kernel='linear', probability=True))
model.fit(X_train, y_train)
```

```
# Predictions
y_score = model.decision_function(X_test)
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# Confusion Matrix
cm = confusion_matrix(
    y_test.argmax(axis=1),
    y_pred.argmax(axis=1)
)

plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Purples')
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

# ----------------------------------------------------------------
# ROC Curve (for one class)
# ----------------------------------------------------------------
fpr, tpr, _ = roc_curve(y_test[:, 0], y_score[:, 0])
roc_auc = auc(fpr, tpr)

plt.figure()
plt.plot(fpr, tpr, label="AUC = %0.2f" % roc_auc)
plt.plot([0, 1], [0, 1], '--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.show()

print("AUC Score:", roc_auc)
```

Program 8

Implement Random Forest ensemble method on a given dataset.

Screenshot



Lab-08

Random Forest

```
import pandas as pd
iris = load_iris()
x = iris.data
y = iris.target
df = pd.dataframe(x, columns = iris.feature_name)
df ['target'] = y
print (df.head())
x_train, xtest, y_test, y_train = train_test_split(
    x, y, test_size = 0.2, random_state = 42)
model = RandomForestClassifier (n_estimator = 100,
random_state = 42)
model.fit ( x_train, y_train)
y_pred = model.predict(x_test)
```

o/p ->

Classification Report

	Precision	recall	f1-score	support
0	1	1	1	10
1	1	1	1	9
2	1	1	1	11
accuracy			1	30
macro avg	1	1	1	30
weighted avg	1	1	1	30

Confusion matrix :

```
[[10  0  0
  0  9  0
  0  0 11]]
```

Code:

```
# Import libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

from google.colab import files
uploaded = files.upload()
data = pd.read_csv("iris (2) - iris (2).csv")
data.head()
```

```python
# Features and target
X = data.drop("species", axis=1)   # change column name if needed
y = data["species"]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ----------------------------------------------------------------------
# 1. Default Random Forest (10 trees)
# ----------------------------------------------------------------------
rf_default = RandomForestClassifier(n_estimators=10, random_state=42)
rf_default.fit(X_train, y_train)

y_pred_default = rf_default.predict(X_test)
default_score = accuracy_score(y_test, y_pred_default)

print("Accuracy with default (10 trees):", default_score)


# 2. Fine-tuning number of trees
tree_range = [5, 10, 20, 50, 100, 200]
best_score = 0
best_n = 0

for n in tree_range:
    rf = RandomForestClassifier(n_estimators=n, random_state=42)
    rf.fit(X_train, y_train)

    y_pred = rf.predict(X_test)
    score = accuracy_score(y_test, y_pred)

    print(f"Trees: {n}, Accuracy: {score}")

    if score > best_score:
        best_score = score
        best_n = n

print("\nBest Accuracy:", best_score)
print("Best Number of Trees:", best_n)
```

```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred)
print(cm)
```

Program 9

Implement Boosting ensemble method on a given dataset.

Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Load dataset
data = pd.read_csv("/content/income.csv")
```

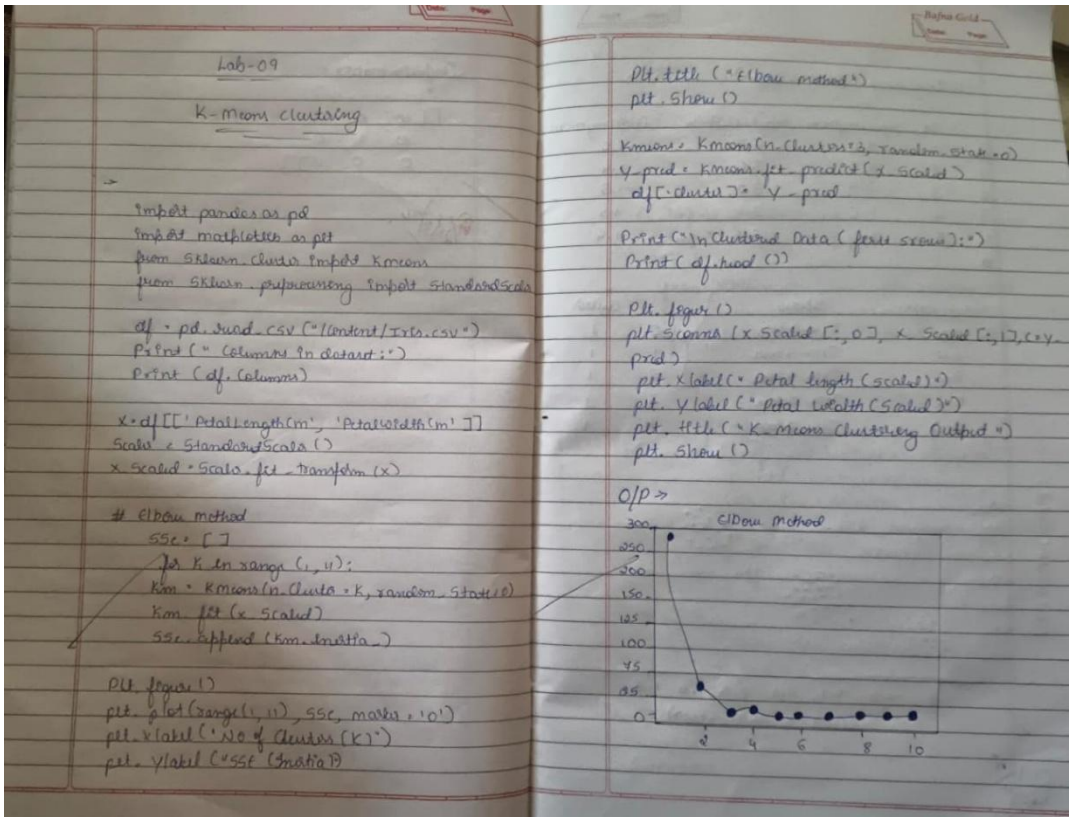
```
data.head()

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Load dataset
data = pd.read_csv("/content/income.csv")


# Handle categorical variables
le = LabelEncoder()
for col in data.columns:
    if data[col].dtype == 'object':
        data[col] = le.fit_transform(data[col])

# Split features and target
X = data.drop("income_level", axis=1)   # change "income" if target column
name differs
y = data["income_level"]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# AdaBoost with Decision Stump
model = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=1),
    n_estimators=10,
    random_state=42
)

# Train
model.fit(X_train, y_train);

# Predict
```

```python
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy with 10 estimators:", accuracy)

results = {}

for n in [5, 10, 20, 50, 100]:
    model = AdaBoostClassifier(
        estimator=DecisionTreeClassifier(max_depth=1),
        n_estimators=n,
        random_state=42
    )

    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    results[n] = acc
    print(f"Estimators: {n}, Accuracy: {acc}")

# Best result
best_n = max(results, key=results.get)
print("\nBest number of estimators:", best_n)
print("Best accuracy:", results[best_n])
```

Program 10

Build k-Means algorithm to cluster a set of data stored in a .CSV file.

Screenshot



Code:

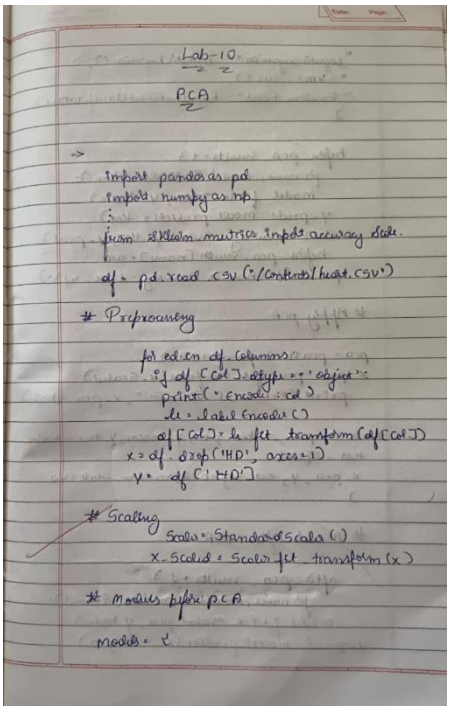
```
from google.colab import files
uploaded = files.upload()


# Import libraries
```

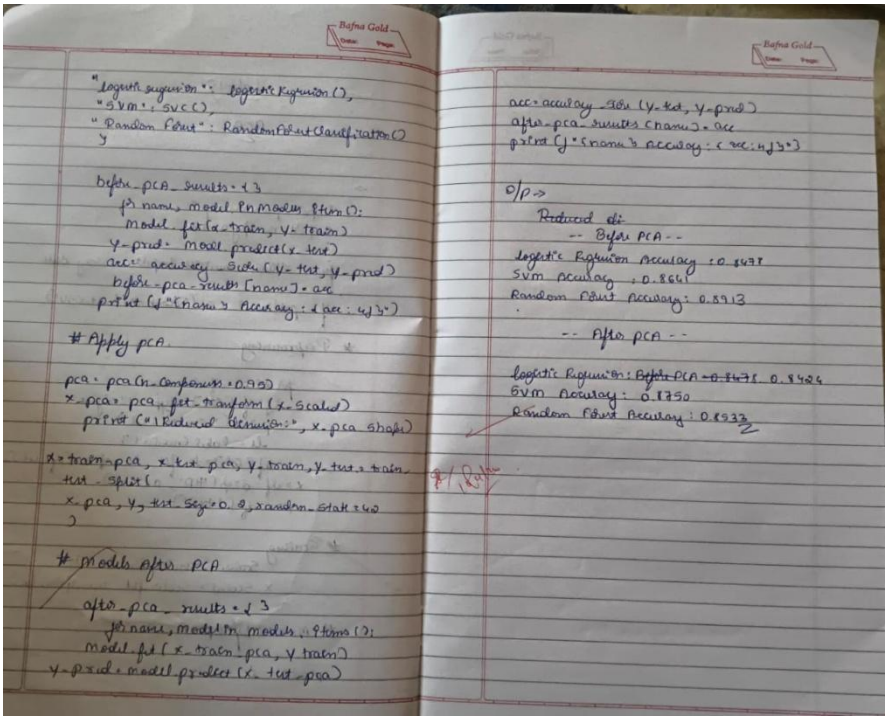
```python
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

# Load dataset
df = pd.read_csv('iris (2) - iris (2).csv')

# Select only petal length and petal width columns
X = df[['petal_length', 'petal_width']]

# Preprocessing: Scale features for better clustering performance
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Elbow method to find optimal K
wcss = []
K_range = range(1, 11)
for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)

# Plot the elbow curve
plt.figure(figsize=(8,5))
plt.plot(K_range, wcss, 'bo--')
plt.title('Elbow Method for Optimal K')
plt.xlabel('Number of clusters (K)')
plt.ylabel('Within-Cluster Sum of Squares (WCSS)')
plt.grid(True)
plt.show()
```

Program 11

Implement Dimensionality reduction using Principle Component Analysis (PCA) method

Screenshot





Code:

```
# ============================================================
# Import Libraries
# ============================================================
import pandas as pd
import numpy as np
```

```
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier

from sklearn.decomposition import PCA

# ============================================================
# Load Dataset
# ============================================================
df = pd.read_csv('/mnt/heart - heart.csv')

print(df.info())
print(df.head())

# ============================================================
# Step 1: Handle Categorical Data
# ============================================================
df = pd.get_dummies(df, drop_first=True)

# ============================================================
# Step 2: Split Features & Target
# ============================================================
X = df.drop('HeartDisease', axis=1)
y = df['HeartDisease']

# ============================================================
# Step 3: Feature Scaling
# ============================================================
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# ============================================================
# Step 4: Train-Test Split
# ============================================================
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)
```

```
# ============================================================
# Step 5: Models BEFORE PCA
# ============================================================
models = {
    "Logistic Regression": LogisticRegression(),
    "SVM": SVC(),
    "Random Forest": RandomForestClassifier()
}

print("Accuracy BEFORE PCA:\n")

before_results = {}

for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    before_results[name] = acc
    print(f"{name}: {acc:.4f}")

# ============================================================
# Step 6: Apply PCA
# ============================================================
pca = PCA(n_components=5)
X_pca = pca.fit_transform(X_scaled)

# Train-Test Split Again
X_train_pca, X_test_pca, y_train, y_test = train_test_split(
    X_pca, y, test_size=0.2, random_state=42
)

# ============================================================
# Step 7: Models AFTER PCA
# ============================================================
print("\nAccuracy AFTER PCA:\n")

after_results = {}

for name, model in models.items():
    model.fit(X_train_pca, y_train)
```

```
    y_pred = model.predict(X_test_pca)
    acc = accuracy_score(y_test, y_pred)
    after_results[name] = acc
    print(f"{name}: {acc:.4f}")

# ============================================================
# Step 8: Comparison
# ============================================================
print("\nComparison:\n")

for name in models.keys():
    print(
        f"{name}: Before = {before_results[name]:.4f}, "
        f"After = {after_results[name]:.4f}"
    )
```